WEB APPLICATION SECURITY ASSESSMENT

Technischer Zwischenbericht · redigierte Fassung

Prüfumfang
Öffentliche Schulwebsite · Schülerportal · Elternportal

Stand: 27. Oktober 2026
Status: Beobachtungen dokumentiert; keine vollständige Sicherheitsfreigabe

Dieser Bericht dokumentiert technische Beobachtungen aus einer begrenzten, überwiegend passiven Prüfung. Er trennt bestätigte Konfigurationen von Scanner-Hinweisen und nicht verifizierten Risiken. Er ist kein Nachweis einer erfolgreichen Kompromittierung.

Die im Bericht enthaltenen Beweisbilder stammen aus den bereitgestellten Original-Screenshots. Detailbereiche mit potenziell sensiblen Headern, Tokens oder identifizierenden Metadaten wurden vorsorglich geschwärzt. Es wurden keine künstlichen Screenshot-Beweise ergänzt.

1. Zusammenfassung

Die Prüfung betrachtete drei öffentlich erreichbare Webanwendungen im Umfeld des Karolinen-Gymnasiums Rosenheim. Der Schwerpunkt lag auf HTTP-Sicherheitsheadern, erkennbaren JavaScript-Abhängigkeiten, Browser-/PWA-Verhalten und einzelnen clientseitigen Codepfaden. Die Ergebnisse zeigen mehrere sinnvolle Härtungsmaßnahmen, jedoch bislang keinen belastbaren Nachweis für unberechtigten Zugriff auf interne Daten oder eine erfolgreich ausgenutzte Schwachstelle.

Wichtigste Beobachtungen

• **Schulwebsite:** In beobachteten Antworten waren unter anderem keine Content-Security-Policy (CSP), kein HSTS und kein Anti-Clickjacking-Header erkennbar. Außerdem wurde jQuery 2.2.4 in einem ausgelieferten JavaScript-Bundle identifiziert.

• **Schülerportal:** HTTP wurde auf HTTPS umgeleitet. Das Portal stellt ein Web-App-Manifest bereit; beobachtetes Service-Worker-/PWA-Verhalten ist für sich genommen normal. Aus den vorliegenden Beobachtungen ergibt sich keine bestätigte Schwachstelle.

• **Elternportal:** Mehrere beobachtete Routen lieferten keine erkennbare CSP. Gleichzeitig waren wichtige Schutzmechanismen vorhanden, darunter HSTS, X-Frame-Options, nosniff, eine restriktive Referrer-Policy sowie ein HttpOnly-/Secure-/SameSite-geschütztes Session-Cookie in der beobachteten Antwort.

• **Abhängigkeiten:** Im Elternportal wurde Bootstrap 3.4.1 festgestellt. Scanner-Zuordnungen zu CVEs wurden nicht unabhängig durch einen Exploit oder eine gezielte Versions-/Codepfadprüfung bestätigt.

• **Clientseitige Logik:** Einzelne JavaScript-Muster verdienen eine gezielte Prüfung auf DOM-XSS und serverseitige Autorisierung. Aus dem sichtbaren Code allein folgt kein Nachweis einer ausnutzbaren Schwachstelle.

Bewertungsskala

Erhöhte Priorität: zuerst prüfen und beheben, wenn der Befund bestätigt ist. **Mittlere Priorität:** sinnvolle Schutzmaßnahme bzw. potenziell relevante Abhängigkeit. **Niedrige Priorität / Hinweis:** Kontext und tatsächliche Auswirkung müssen geprüft werden. Scanner-Schweregrade sind keine automatische Aussage über reale Ausnutzbarkeit.

2. Umfang und Methodik

Betrachtete Ziele:

• https://www.karolinen-gymnasium-rosenheim.de/ — öffentliche Schulwebsite

• https://schueler.schule-infoportal.de/login — Schülerportal

• https://karogyro.eltern-portal.org/ — Elternportal

Die Auswertung stützt sich auf bereitgestellte Screenshots aus OWASP ZAP/Browser-Inspektion sowie die in der laufenden Analyse dokumentierten HTTP-Antworten und Codebeobachtungen. Die Prüfung war begrenzt und nicht destruktiv. Es wurden keine SQL-Injection-Angriffe erfolgreich nachgewiesen; SQL-Injection wurde nicht als bestätigte Schwachstelle bewertet. Keine Brute-Force-, Last-, destruktiven oder fremde Systeme betreffenden Tests werden hier als durchgeführt dargestellt.

Grenzen:

• Ein fehlender Header ist zunächst eine Härtungsbeobachtung; er beweist nicht automatisch eine ausnutzbare Schwachstelle.

• Ein Scanner-Hinweis auf eine Bibliotheksversion beweist nicht, dass der verwundbare Codepfad erreichbar ist.

• Serverseitige Autorisierung, CSRF-Wirkung, Upload-Validierung und DOM-XSS-Ausnutzbarkeit wurden nicht abschließend verifiziert.

• Es wird keine Freigabe oder Autorisierung durch Schulleitung behauptet.

3. Befunde – Schulwebsite

3.1 Content Security Policy nicht erkennbar

Beobachtung: In den betrachteten Antworten der Schulwebsite war kein Content-Security-Policy-Header erkennbar. CSP kann einschränken, aus welchen Quellen Skripte, Styles, Bilder und weitere Ressourcen geladen werden dürfen, und bietet zusätzliche Verteidigung gegen bestimmte Cross-Site-Scripting-Szenarien.

Einordnung: Dies ist eine fehlende Schutzschicht, kein eigenständiger Nachweis für XSS. Ob ein konkreter Angriff möglich wäre, hängt unter anderem von der Ausgabe-Kodierung, vorhandenen Skriptquellen und den verwendeten Endpunkten ab.

Empfehlung: Eine zunächst report-only getestete CSP erstellen, erlaubte Ressourcen inventarisieren und anschließend schrittweise erzwingen. Inline-Skripte möglichst über Nonces/Hashes absichern und Richtlinien auf benötigte Quellen begrenzen.

3.2 Veraltete jQuery-Version erkannt

Beobachtung: In `/_assets/a92153751098915699a1afa17e77f864/JavaScripts/main-bbf91142.js` wurde jQuery 2.2.4 identifiziert. Diese Version ist deutlich älter als aktuelle gepflegte Releases. In der Analyse wurden CVE-2020-11022 und CVE-2020-11023 als relevante Hinweise zu jQuery genannt.

Einordnung: Die Versionsfeststellung ist der belastbare Teil des Befunds. Die tatsächliche Ausnutzbarkeit wurde nicht nachgewiesen; sie hängt von den konkret verwendeten jQuery-APIs und davon ab, ob nicht vertrauenswürdige Inhalte in entsprechende DOM-Manipulationen gelangen.

Empfehlung: Abhängigkeit auf eine unterstützte jQuery-Version aktualisieren, Regressionstests durchführen und die Nutzung von HTML-parsenden APIs mit nicht vertrauenswürdigen Eingaben prüfen.

3.3 HSTS nicht erkennbar

Beobachtung: In den betrachteten Antworten wurde kein `Strict-Transport-Security`-Header erkannt. Gleichzeitig war HTTPS erreichbar; das Vorhandensein einer HTTPS-Weiterleitung allein ersetzt HSTS nicht.

Empfehlung: Nach Prüfung sämtlicher Subdomains HSTS mit angemessener Laufzeit aktivieren. `includeSubDomains` und Preload nur verwenden, wenn alle betroffenen Hosts dauerhaft HTTPS unterstützen.

3.4 Anti-Clickjacking-Schutz nicht erkennbar

Beobachtung: In einer betrachteten Antwort war kein `X-Frame-Options`-Header erkennbar. Eine CSP-Direktive `frame-ancestors` wurde nicht separat bestätigt.

Einordnung und Empfehlung: Das Fehlen ist ein Härtungshinweis, kein Beweis für einen konkreten Clickjacking-Angriff. Wenn Einbettung nicht benötigt wird, `frame-ancestors 'none'` bzw. eine passende restriktive Richtlinie setzen; alternativ kompatible X-Frame-Options-Regel ergänzen.

3.5 Öffentliche Ressourcen und Cache-Header

Für eine öffentliche Schriftdatei wurde `Access-Control-Allow-Origin: *` beobachtet; ein Credential-Allow-Header war nicht vorhanden. Bei einer öffentlich ausgelieferten statischen Schrift ist dies häufig erwartbar und stellt allein keinen bestätigten Sicherheitsfehler dar. Die Homepage zeigte Cache-Header mit einer Laufzeit von rund 3,5 Stunden; öffentliche statische Medien zeigten längere Cache-Zeiten. Solche Werte können normal sein und sollten nach Sensibilität der Inhalte bewertet werden.

3.6 Kommentare und Zeitstempel

In untersuchten JavaScript-Dateien sichtbare Kommentare wirkten wie gewöhnliche Bibliotheks-/Quellkommentare; es wurde darin kein bestätigtes Geheimnis dokumentiert. Zeitstempel und TYPO3-Cache-Metadaten geben technische Informationen preis, sind aber für sich genommen kein Nachweis einer Sicherheitslücke. Debug-/Cache-Header sollten dennoch auf notwendige Offenlegung geprüft werden.

4. Befunde – Schülerportal

4.1 HTTPS-Weiterleitung und PWA-Verhalten

Die HTTP-URL leitete auf HTTPS weiter. Das Portal stellte ein Web-App-Manifest bereit; beobachtete Service-Worker-Aktivierung und -Beendigung sind typische PWA-Lebenszyklusereignisse. Diese Beobachtungen sind keine Schwachstellen an sich.

4.2 Header-Bewertung eingeschränkt

Bei der Login-Ressource wurde eine 304-Not-Modified-Antwort beobachtet. Eine solche Antwort enthält nicht notwendigerweise alle Header, die in einer vollständigen 200-Antwort erscheinen würden. Daher lässt sich daraus allein nicht zuverlässig ableiten, dass Schutzheader fehlen. Eine saubere Bewertung sollte die vollständige Antwort nach Cache-Neuladung und die tatsächlich ausgelieferte Seite vergleichen.

4.3 Ergebnisstatus

Auf Basis der vorliegenden Beobachtungen ist für das Schülerportal keine konkrete ausnutzbare Schwachstelle bestätigt. Die vorhandene Installations-/PWA-Dokumentation beschreibt Funktionalität, nicht einen Sicherheitsmangel. Weitere Tests sollten nur innerhalb eines klar festgelegten und erlaubten Prüfumfangs erfolgen.

5. Befunde – Elternportal

5.1 Content Security Policy nicht erkennbar

In den beobachteten Antworten für mehrere Routen (unter anderem Index/Login, Passwort-vergessen, Registrierung und CAPTCHA-Endpunkt) war kein CSP-Header erkennbar. Diese Meldungen werden hier als ein übergreifender Konfigurationsbefund zusammengefasst, nicht als vier voneinander unabhängige Schwachstellen.

Positiv beobachtet: Die Login-Antwort enthielt `Cache-Control: no-store, no-cache, must-revalidate`, `X-Frame-Options: SAMEORIGIN`, `Referrer-Policy: strict-origin-when-cross-origin`, `X-Content-Type-Options: nosniff` sowie HSTS mit einer Laufzeit von einem Jahr und `includeSubDomains`. Das Session-Cookie war in der beobachteten Antwort mit Secure, HttpOnly und SameSite=Strict markiert. Konkrete Cookie-Werte werden absichtlich nicht dokumentiert.

5.2 Bootstrap 3.4.1 – veraltete Abhängigkeit

In `/includes/project/js/bootstrap.min.js` wurde Bootstrap 3.4.1 festgestellt. Der Scanner nannte CVE-2025-1647 und CVE-2024-6485; diese Zuordnungen wurden in dieser Prüfung nicht unabhängig verifiziert. Eine alte Bibliotheksversion ist ein Wartungs- und Risikohinweis, aber ohne Abgleich des konkreten Codes und erreichbarer Komponenten kein Nachweis erfolgreicher Ausnutzung.

Empfehlung: Auf eine unterstützte Bootstrap-Version migrieren, Komponentenabhängigkeiten und mögliche Breaking Changes prüfen sowie nicht benötigte Plugins entfernen.

5.3 DOM-XSS als Prüfhinweis – nicht bestätigt

In einem JavaScript-Codepfad wurden unter anderem Aufrufe beobachtet, die Werte aus einem auslösenden Element lesen und mit jQuery `.html()` in Elemente wie `#rep_time`, `#warning` oder einen Titelbereich schreiben. Das kann riskant sein, wenn die eingesetzten Werte durch Angreifer kontrollierbar sind und HTML nicht sicher begrenzt wird. Es ist bislang nicht belegt, dass diese Werte tatsächlich aus einer nicht vertrauenswürdigen Quelle stammen oder dass sich JavaScript ausführen lässt.

Empfehlung: Für reine Textwerte `.text()` statt `.html()` verwenden; Datenquellen nachvollziehen und HTML nur mit geeigneter Sanitization zulassen. Eine CSP als zusätzliche Schutzschicht ergänzen.

5.4 Zustandsänderung über GET – Prüfhinweis

Ein sichtbarer JavaScript-Pfad rief `api/elternbrief_bestaetigen.php?eb=...` per GET auf und aktualisierte anschließend die Oberfläche. Wenn dieser Endpunkt serverseitig einen Bestätigungszustand ändert, wäre GET als zustandsändernde Methode ungünstig und könnte CSRF-Risiken schaffen. Aus dem Client-Code allein ist nicht

bewiesen, dass der Endpunkt tatsächlich ohne weitere Schutzprüfung einen Zustand ändert.

Empfehlung: Zustandsänderungen über POST/PUT mit serverseitiger CSRF-Prüfung durchführen und Authentisierung sowie Berechtigung auf dem Server prüfen.

5.5 Kind-/Datensatz-Auswahl und Autorisierung – Prüfhinweis

Ein JavaScript-Pfad sendet eine ausgewählte ID an `/api/set_child.php?id=...`. Der Server muss sicherstellen, dass die angefragte ID zum angemeldeten Benutzer gehört. Es wurde ausdrücklich kein Zugriff auf fremde IDs getestet oder nachgewiesen.

Empfehlung: Bei jeder Anfrage serverseitig die Beziehung zwischen Session, Benutzer und Datensatz prüfen; niemals allein auf clientseitige IDs oder ausgeblendete Felder vertrauen.

6. Weitere Beobachtungen und Grenzen

6.1 Registrierung, CAPTCHA und Passwort-Reset

Die Registrierungsseite zeigte Felder für PIN/Benutzerkennung, eine CAPTCHA-Grafik sowie clientseitige Prüfungen der übereinstimmenden E-Mail-Eingaben. Der Passwort-Reset stellte ein E-Mail-Feld und ein verstecktes CSRF-Feld bereit. Das Vorhandensein dieser Kontrollen beweist nicht, dass die serverseitige Validierung korrekt ist; ein Umgehen oder Ausnutzen wurde nicht bestätigt. CAPTCHA-Bilddaten, CSRF-Werte und Formulardaten werden in diesem Bericht nicht wiedergegeben.

6.2 Dateiupload – Größenbegrenzung

Im sichtbaren Client-Code wurde eine Prüfung der ausgewählten Gesamtdateigröße auf 16 MB festgestellt. Clientseitige Prüfungen lassen sich grundsätzlich umgehen und ersetzen keine serverseitige Begrenzung. Es wurde nicht nachgewiesen, dass die Serverprüfung fehlt.

6.3 Externe CDN-Ressourcen

Für einzelne externe Azure-CDN-Ressourcen wurden Scanner-Hinweise zu CORS bzw. `X-Content-Type-Options` angezeigt. Da es sich um Drittanbieter-Hosts handelt und deren Einordnung/Verantwortlichkeit nicht abschließend geklärt wurde, werden diese Hinweise nicht als bestätigte Schwachstellen der Schul- oder Portalserver gewertet. Wildcard-CORS bei öffentlich abrufbarer Konfiguration kann beabsichtigt sein; der konkrete Dateninhalt und die Vertrauensgrenzen sind entscheidend.

6.4 SQL-Injection

Es liegt kein bestätigter SQL-Injection-Befund vor. SQL-Injection wurde nicht als erfolgreich getestet oder ausgenutzt dokumentiert. Dieser Bericht behauptet weder Datenbankzugriff noch Zugriff auf interne Schüler-/Elterndaten.

7. Priorisierte Maßnahmen

Priorität	Maßnahme	Begründung
Hoch / zeitnah	jQuery und Bootstrap auf unterstützte Versionen aktualisieren; betroffene Nutzungspfade testen.	Veraltete clientseitige Abhängigkeiten reduzieren; Ausnutzbarkeit aktuell nicht bestätigt.
Hoch / zeitnah	CSP für Schulwebsite und Elternportal konzipieren, zunächst im Report-Only-Modus testen, danach erzwingen.	Zusätzliche Begrenzung von Skriptquellen und Inline-Ausführung.
Mittel	HSTS für Schulwebsite prüfen und aktivieren, nachdem Subdomain-Kompatibilität geklärt ist.	Schutz gegen Downgrade-/SSL-Stripping-Szenarien nach Erstkontakt.
Mittel	Frame-Einbettung gezielt konfigurieren; frame-ancestors/X-Frame-Options konsistent setzen.	Clickjacking-Risiko verringern.
Mittel	DOM-Ausgaben auf .html()-Nutzung prüfen; reine Textwerte mit .text() setzen.	Verhindert, dass untrusted Werte als HTML interpretiert werden.
Mittel	Serverseitige Autorisierung für Kind-/Datensatz-IDs, CSRF-Schutz für Zustandsänderungen und Uploadlimits überprüfen.	Clientseitiger Code kann serverseitige Sicherheitskontrollen nicht ersetzen.
Niedrig / Hygiene	Nicht erforderliche Debug-/Cache-Header und Metadaten reduzieren; Cache-Regeln je Inhaltstyp überprüfen.	Verringert unnötige technische Offenlegung und verhindert Fehlkonfigurationen.

8. Beweisbilder aus den Original-Screenshots

Die folgenden Bilder stammen aus den bereitgestellten Originalaufnahmen der Prüfoberfläche. Zur Veröffentlichung wurden die großen Detailbereiche geschwärzt, in denen Header, URLs, Token, Cookies oder sonstige sensible Werte

enthalten sein könnten. Die Schwärzung ist bewusst konservativ; dadurch werden manche technischen Details nicht mehr direkt lesbar. Für eine vertrauliche technische Meldung kann eine separate, stärker detaillierte Fassung erstellt werden.

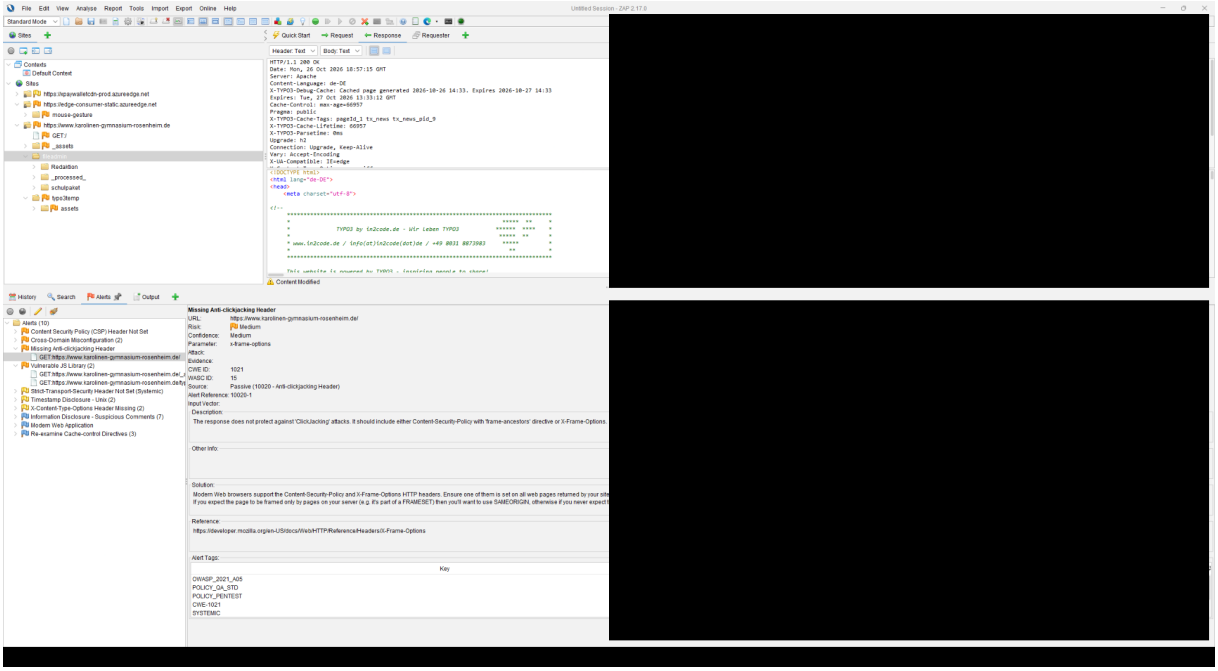


Abbildung 1 – Originalaufnahme der Prüfoberfläche; Detailpaneele geschwärzt.

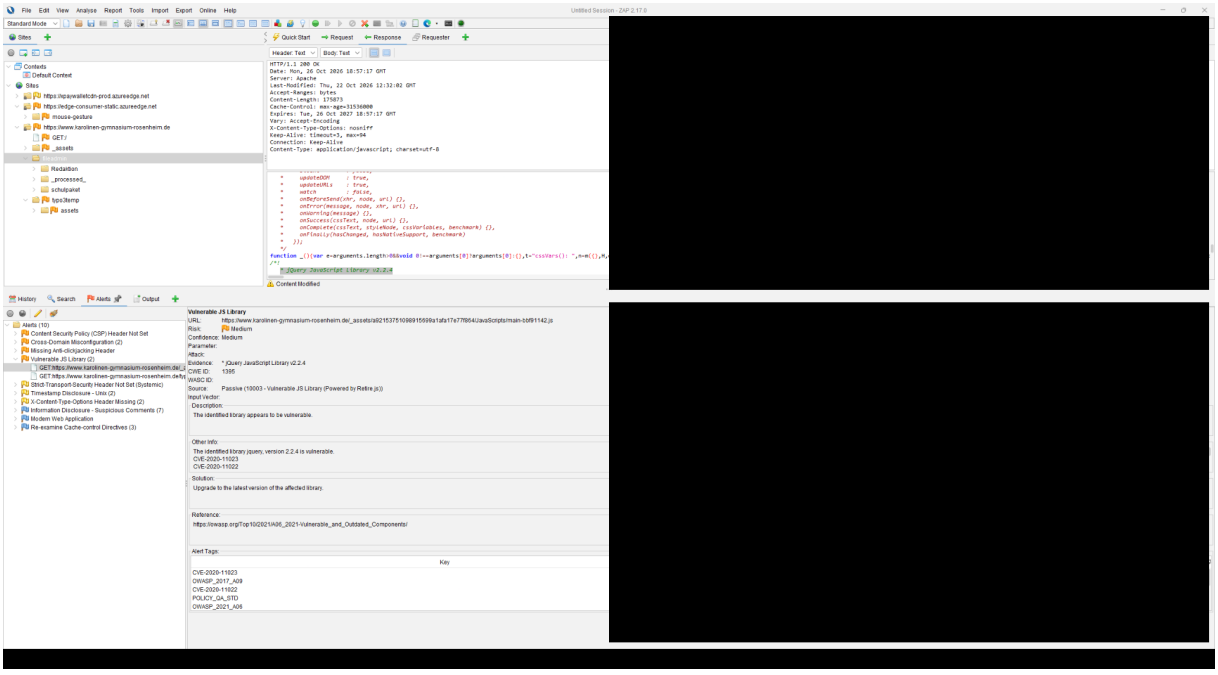


Abbildung 2 – Originalaufnahme mit ausgewähltem Befund; potenziell sensible Details geschwärzt.

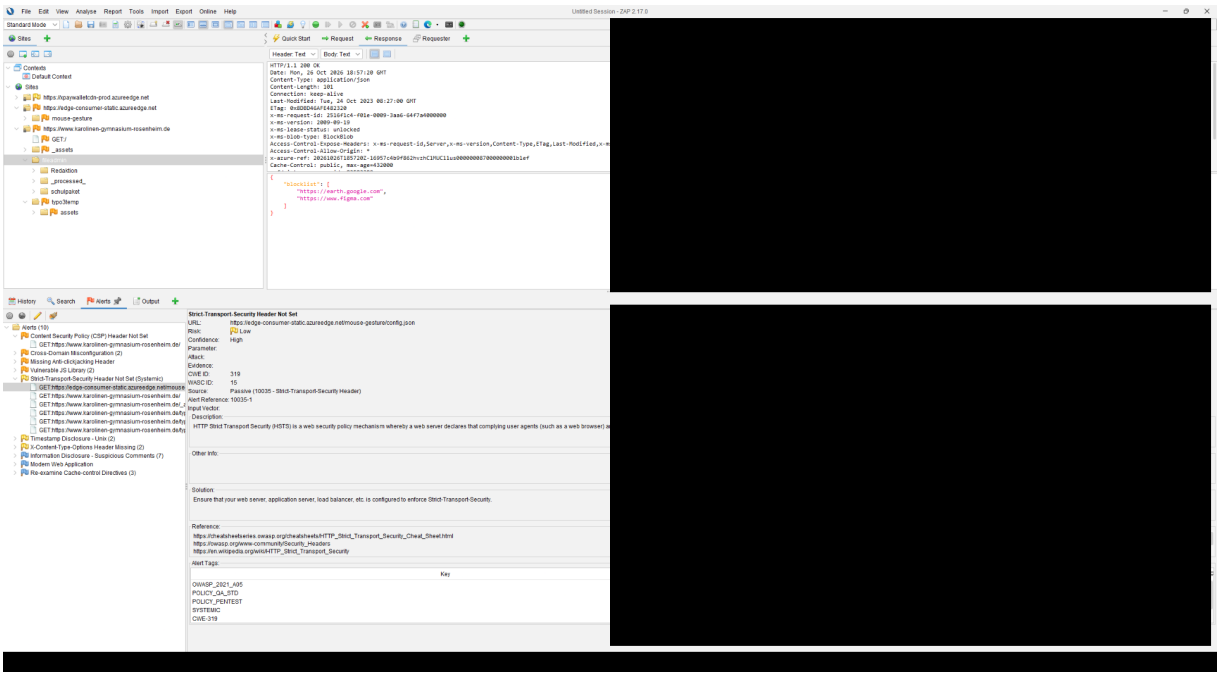


Abbildung 3 – Originalaufnahme der HTTP-/Scanner-Ansicht; Detailpaneele geschwärzt.

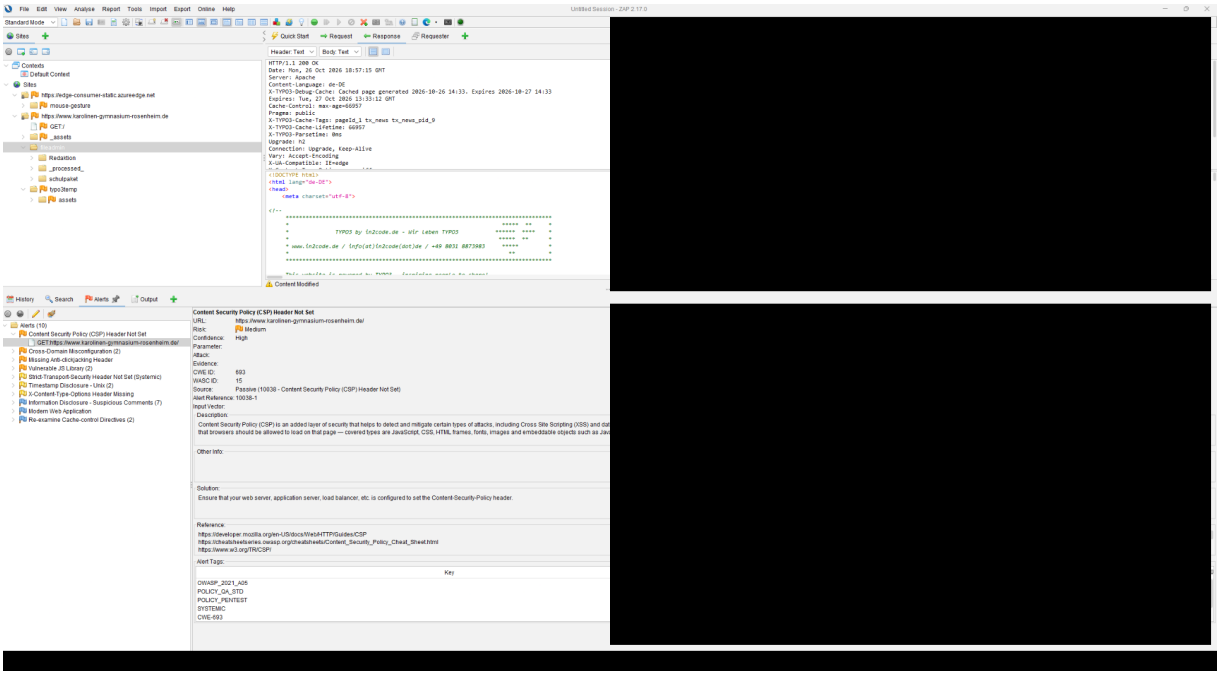


Abbildung 4 – Originalaufnahme der Response-/Header-Ansicht; Wertebereich geschwärzt.

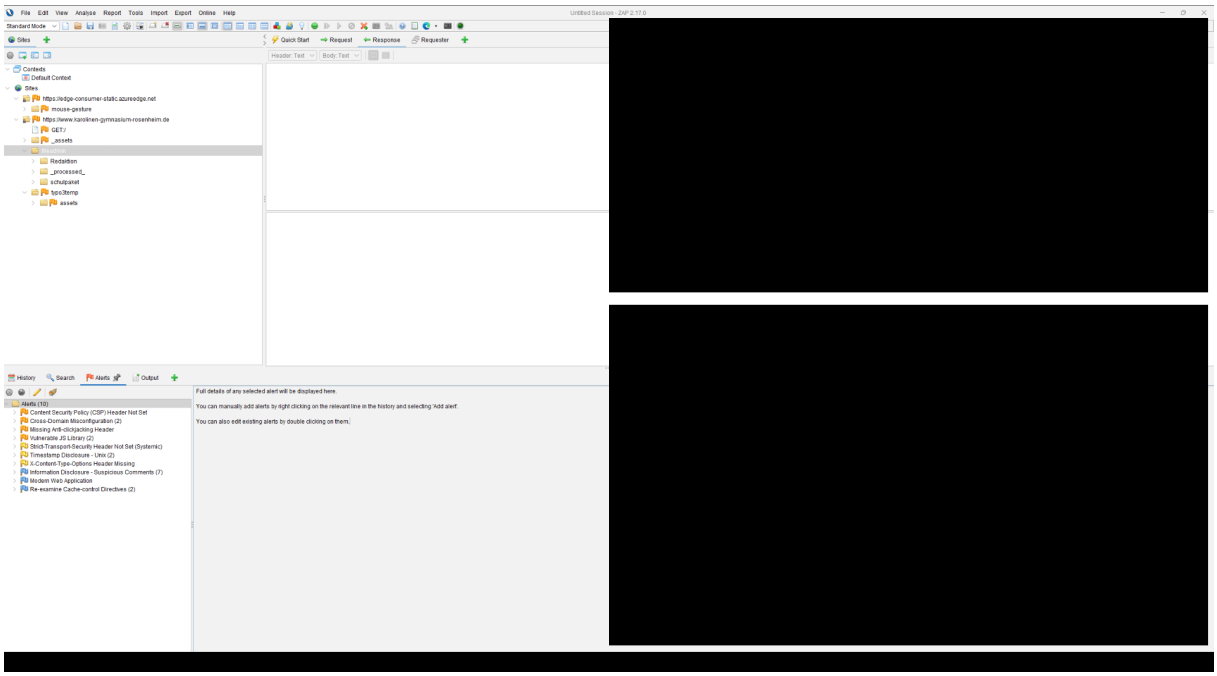


Abbildung 5 – Zusätzliche Originalaufnahme der Analyseoberfläche; Detailpaneele geschwärzt.

9. Methodische Hinweise und Schlussfolgerung

Die Ergebnisse sind als begrenzte technische Bestandsaufnahme zu verstehen. Die stärksten belegten Punkte sind die festgestellten Bibliotheksversionen und die in den untersuchten Antworten sichtbaren Header-Konfigurationen. Andere Themen – insbesondere DOM-XSS, CSRF bei Zustandsänderungen, Objektberechtigungen und serverseitige Uploadprüfung – sind nachvollziehbare Prüfhinweise, aber noch keine bestätigten Schwachstellen.

Es wird empfohlen, die bestätigten Konfigurations- und Abhängigkeitsbefunde zuerst durch die zuständigen Betreiber prüfen zu lassen. Anschließend können die Prüfhinweise in einem abgestimmten Testfenster mit klar definiertem Umfang verifiziert werden. Bis dahin sollten sie im Bericht als offen bzw. nicht bestätigt markiert bleiben.

Veröffentlichungshinweis

• Diese Fassung enthält keine Session-Cookie-Werte, CSRF-Tokens, Zugangsdaten oder CAPTCHA-Daten.

• Es werden keine privaten Namen, E-Mail-Adressen oder internen Kontokennungen als Beweis veröffentlicht.

• Die Screenshots sind echte bereitgestellte Aufnahmen; keine KI-generierten Beweisbilder.

• Vor einer öffentlichen Veröffentlichung sollten auch PDF-Metadaten, Dateinamen, Links, Screenshots und eingebettete Anmerkungen erneut auf Identifikatoren geprüft werden.

Endstatus: Mehrere Härtungsmaßnahmen und weiter zu verifizierende Codepfade wurden identifiziert. Eine erfolgreiche Kompromittierung, SQL-Injection oder unberechtigter Zugriff auf interne Daten ist mit dem vorliegenden Material nicht belegt.